scientific reports

Check for updates

OPEN

Integrating self-attention and LSTM into TD3 for robust mobile robot navigation in dynamic environments

Luyun Chen[1], Qiang Tang[2], Rui Xu[3] & You Chen[4]✉

Mobile robot path planning in dynamic environments is challenging because existing deep reinforcement learning methods lack temporal memory, suffer from inefficient sample utilization under uniform replay, and face credit assignment difficulties with sparse rewards. This paper proposes the Self-Attention LSTM TD3 (SAL-TD3) algorithm, which integrates LSTM networks and multi-head self-attention into the TD3 framework to capture temporal dependencies for proactive obstacle avoidance. A rank-based prioritized experience replay with n-step returns improves sample efficiency, and a composite reward function provides dense feedback for efficient policy learning. Experiments show that SAL-TD3 achieves a 91% success rate (vs. 77% for TD3), reduces path length by 16.6%, and lowers collision rate from 23% to 9%. Generalization tests and real-world robot deployment confirm robust sim-to-real transfer performance.

Keywords Mobile robot path planning, Deep reinforcement learning, Twin Delayed Deep Deterministic Policy Gradient, Self-attention mechanism

Mobile robot path planning in dynamic environments constitutes a fundamental challenge in autonomous systems[1], with broad applications spanning warehouse logistics, hospital service robots, autonomous vehicles, and search-and-rescue operations[2]. The ability to navigate safely and efficiently while avoiding both static obstacles and unpredictably moving agents such as pedestrians directly determines the practical deployment feasibility of mobile robots in human-populated spaces[3]. As service robots increasingly operate in close proximity to humans in shopping malls, airports, and healthcare facilities, the demand for robust navigation algorithms capable of real-time adaptation to dynamic environmental changes has become critically important[4]. Unlike controlled industrial settings where robot trajectories can be pre-programmed, real-world navigation requires continuous decision-making under uncertainty, where the consequences of navigation failures range from task inefficiency to potential safety hazards for both robots and humans[5].

Traditional path planning algorithms, including A*, Dijkstra, RRT, and their variants, have achieved considerable success in static environments by computing optimal or near-optimal paths based on known map representations[6]. However, these approaches fundamentally rely on the assumption that obstacle configurations remain fixed during path execution, rendering them inadequate for environments with moving obstacles[7]. When pedestrians or other dynamic agents intersect the planned trajectory, traditional methods must trigger complete replanning cycles, introducing computational delays that compromise real-time responsiveness[8]. Furthermore, these algorithms lack the capability to anticipate obstacle motion patterns, resulting in purely reactive avoidance behaviors that often produce inefficient detour paths or, in confined spaces, navigation failures[9]. The integration of prediction modules with traditional planners has been explored, yet such hybrid approaches suffer from error accumulation between separate prediction and planning components, limiting their effectiveness in highly dynamic scenarios[10].

Deep reinforcement learning has emerged as a promising paradigm for addressing the limitations of traditional planning methods by learning end-to-end navigation policies directly from sensory observations[11]. Actor-critic algorithms such as DDPG, TD3, and SAC enable continuous action control essential for smooth robot motion,

[1]College of Information Science and Engineering, Northeastern University, Shenyang 110819, China. [2]School of Artificial Intelligence, Anhui University of Science and Technology, Hefei 231131, China. [3]School of Engineering and Applied Science, Princeton University, Princeton, NJ, USA. [4]School of Housing, Building and Planning, Universiti Sains Malaysia, Penang 11700, Malaysia. ✉email: vxr4175@163.com

while the learned policies can implicitly capture environment dynamics through interaction experience[12]. Nevertheless, current deep reinforcement learning approaches for robot navigation exhibit several critical deficiencies that hinder their practical deployment. First, standard feedforward neural network architectures process each observation independently without maintaining memory of historical states, fundamentally limiting the agent's ability to infer obstacle motion trajectories and anticipate future positions[13]. This memoryless property results in reactive rather than predictive avoidance behaviors, particularly problematic when navigating among multiple moving obstacles with diverse motion patterns. Second, uniform experience replay sampling treats all transitions as equally informative, leading to inefficient utilization of valuable experiences such as successful obstacle avoidance maneuvers or near-collision events that contain rich learning signals[14]. This sample inefficiency prolongs training time and may result in suboptimal policies that fail to adequately learn from rare but critical situations. Third, single-step temporal-difference learning propagates reward signals slowly through trajectories, causing credit assignment difficulties in navigation tasks where goal-reaching rewards are sparse and delayed[15]. The combination of these limitations manifests as slow convergence, unstable training dynamics, and degraded generalization performance when deployed in environments differing from training conditions[16].

To address the aforementioned challenges, this paper proposes the Self-Attention LSTM TD3 (SAL-TD3) algorithm, a novel deep reinforcement learning framework that synergistically integrates temporal modeling, attention mechanisms, and prioritized learning for enhanced mobile robot path planning. The proposed approach incorporates LSTM networks into both actor and critic architectures to maintain memory of historical observations, enabling the agent to capture temporal dependencies and implicitly predict obstacle motion patterns for proactive avoidance decisions. A multi-head self-attention mechanism complements the LSTM processing by allowing the network to selectively focus on relevant temporal segments, effectively combining the sequential processing strengths of recurrent networks with the long-range dependency modeling capabilities of attention mechanisms[17]. The framework further employs prioritized experience replay with rank-based prioritization to concentrate learning on high-value transitions, substantially improving sample efficiency and accelerating convergence. Additionally, n-step returns are integrated to enable more effective credit assignment by propagating reward information across multiple time steps, addressing the sparse reward challenge inherent in goal-reaching navigation tasks. A carefully designed composite reward function incorporating heading alignment, distance progress, and obstacle safety components provides dense feedback signals throughout navigation episodes, guiding the agent toward efficient and safe navigation behaviors.

It is worth discussing why the combination of LSTM and multi-head self-attention represents the most suitable architectural choice for mobile robot navigation in dynamic environments compared with alternative temporal modeling strategies. First, pure feedforward architectures (e.g., standard multilayer perceptrons) lack any mechanism for retaining historical information, rendering them fundamentally incapable of inferring obstacle motion trajectories from sequential observations. Second, Transformer-only architectures, while powerful for capturing long-range dependencies through global self-attention, require fixed-length context windows and incur quadratic computational complexity with respect to sequence length, making them less suitable for the online streaming nature of robotic control where observations arrive continuously and inference latency must remain within single-digit milliseconds. Third, convolutional temporal models such as Temporal Convolutional Networks (TCN) can capture local temporal patterns efficiently but have limited receptive fields and cannot selectively retain or forget information over varying time horizons as the dynamics of the environment change. In contrast, LSTM networks are specifically designed with gating mechanisms (forget gate, input gate, output gate) that enable selective retention and forgetting of information across time steps, providing a compact and adaptive temporal memory well suited to the variable-duration interactions encountered during dynamic navigation. However, LSTM alone compresses the entire observation history into a fixed-size hidden state, which can become an information bottleneck when the agent must attend to specific past events—for instance, a pedestrian's sudden direction change several steps earlier. The multi-head self-attention mechanism directly addresses this limitation by enabling the network to compute relevance-weighted combinations over the LSTM output sequence, selectively amplifying the most informative temporal segments while suppressing irrelevant ones. This complementary integration thus combines the sequential memory strengths of LSTM with the selective long-range dependency modeling of self-attention. Our ablation study in section "Experimental analysis" empirically validates this design: removing the LSTM component causes the largest performance degradation (success rate drops from 91% to 83%), while removing self-attention also leads to a measurable 3 percentage point decrease, confirming the necessity of their joint deployment.

The main contributions of this paper are summarized as follows:

- We propose the SAL-TD3 algorithm that integrates LSTM-based temporal modeling with multi-head self-attention mechanisms into the TD3 framework, enabling the agent to capture sequential dependencies in dynamic environments and selectively attend to relevant historical observations for improved obstacle motion prediction and proactive avoidance decision-making.
- We develop an enhanced experience replay mechanism combining rank-based prioritized sampling with n-step return computation, which significantly improves sample efficiency by focusing learning on informative transitions while providing more accurate value estimates through multi-step bootstrapping for effective credit assignment in sparse reward navigation scenarios.
- We design a composite reward function with heading alignment, distance progress, and obstacle safety components that provides dense and informative feedback signals, facilitating efficient policy learning while balancing navigation efficiency with collision avoidance requirements.
- We conduct comprehensive experiments including simulation studies in dynamic environments with multiple moving obstacles, generalization tests in unseen configurations, and real-world robot deployment valida-

tion, demonstrating that SAL-TD3 achieves 91% success rate compared to 77% for baseline TD3, with 16.6% reduction in path length and robust sim-to-real transfer performance.

Related work

Traditional path planning algorithms

Classical path planning approaches can be broadly categorized into graph-based methods and sampling-based methods[18]. The A* algorithm represents a foundational graph-based approach that guarantees optimal path finding through heuristic-guided search, widely adopted in grid-based navigation scenarios[19]. Dijkstra's algorithm provides optimal solutions without heuristic guidance but suffers from computational inefficiency in large-scale environments[20]. The D* algorithm and its variant D* Lite extend these approaches to handle dynamic environments through incremental replanning capabilities[21].

Sampling-based methods, including Rapidly-exploring Random Trees (RRT) and its asymptotically optimal variant RRT*, have gained prominence for high-dimensional configuration spaces due to their probabilistic completeness guarantees[22]. The Probabilistic Roadmap Method (PRM) constructs reusable roadmaps for multi-query planning scenarios[23]. However, these traditional approaches fundamentally assume static or known obstacle configurations and lack mechanisms for real-time adaptation to dynamic environments, limiting their applicability in scenarios involving moving obstacles such as pedestrians or other mobile agents[24].

Deep reinforcement learning for robot navigation

The emergence of deep reinforcement learning has revolutionized autonomous navigation by enabling end-to-end learning of navigation policies directly from sensory inputs[25]. Deep Q-Network (DQN) demonstrated the feasibility of combining deep neural networks with reinforcement learning for high-dimensional state spaces, though its discrete action space limitation restricts applicability to continuous robot control tasks[26].

For continuous action spaces essential in mobile robot control, the Deep Deterministic Policy Gradient (DDPG) algorithm introduced an actor-critic architecture that learns deterministic policies through off-policy training[27]. However, DDPG suffers from overestimation bias in value function approximation and training instability, particularly in complex navigation environments[28]. The Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm addresses these limitations through three key modifications: clipped double Q-learning to mitigate overestimation, delayed policy updates to reduce per-update error, and target policy smoothing for improved robustness[29]. These enhancements have established TD3 as a preferred baseline for continuous control tasks in robotics[30].

The Soft Actor-Critic (SAC) algorithm presents an alternative approach based on maximum entropy reinforcement learning, encouraging exploration through entropy regularization while maintaining sample efficiency[31]. Comparative studies have demonstrated that both TD3 and SAC achieve competitive performance across various robotic manipulation and navigation benchmarks, with TD3 often exhibiting more stable training dynamics in deterministic policy scenarios[32].

Recurrent neural networks for temporal modeling

Capturing temporal dependencies in sequential decision-making tasks has motivated the integration of recurrent neural network architectures into reinforcement learning frameworks[33]. Long Short-Term Memory (LSTM) networks address the vanishing gradient problem inherent in vanilla recurrent networks through gating mechanisms that selectively retain or discard information across time steps[34]. The application of LSTM to deep reinforcement learning was pioneered by Hausknecht and Stone, who demonstrated that recurrent architectures enable agents to handle partial observability by maintaining memory of past observations[35].

Prioritized experience replay

Experience replay constitutes a fundamental technique in off-policy reinforcement learning, enabling sample-efficient learning by storing and reusing past transitions[36]. The original formulation employs uniform random sampling from the replay buffer, which treats all experiences as equally informative regardless of their learning potential[37].

Prioritized Experience Replay (PER) improves upon uniform sampling by assigning sampling probabilities proportional to the temporal-difference (TD) error magnitude, ensuring that transitions with higher prediction errors receive increased attention during training[38]. This prioritization mechanism accelerates learning by focusing computational resources on the most informative experiences. The framework incorporates importance sampling weights to correct for the bias introduced by non-uniform sampling, with an annealing schedule that gradually increases correction strength as training progresses[39].

Subsequent research has explored various prioritization strategies, including rank-based prioritization for robustness across different reward scales, and curiosity-driven prioritization for exploration in sparse reward environments[40]. The integration of PER with actor-critic algorithms has demonstrated consistent improvements in sample efficiency and final policy performance across diverse continuous control benchmarks[41].

Attention mechanisms in reinforcement learning

Attention mechanisms, originally developed for sequence-to-sequence modeling in natural language processing, have been increasingly adopted in reinforcement learning to enable selective focus on relevant features or temporal segments[42]. The self-attention mechanism computes attention weights based on query-key similarity, allowing networks to dynamically weigh the importance of different input elements[43].

In robotic applications, attention mechanisms have been employed for multi-modal sensor fusion, enabling agents to selectively attend to relevant sensory modalities based on task requirements[44]. For temporal modeling, self-attention provides a complementary mechanism to recurrent processing by capturing long-range

dependencies through direct attention over the entire sequence history, mitigating the information bottleneck inherent in fixed-size hidden states[45].

Recent work has explored the combination of attention mechanisms with LSTM networks for enhanced temporal modeling. The Transformer architecture demonstrates that self-attention alone can achieve competitive performance with recurrent models while enabling parallel computation[46]. However, for online decision-making in robotics where observations arrive sequentially, hybrid architectures combining LSTM's sequential processing with attention's selective focusing have shown promising results[47].

N-step returns and multi-step learning

The choice of bootstrapping horizon in temporal-difference learning presents a fundamental trade-off between bias and variance in value estimation[48]. Single-step TD learning exhibits low variance but high bias due to reliance on potentially inaccurate value function estimates, while Monte Carlo methods provide unbiased estimates at the cost of high variance from trajectory-level returns[49].

N-step returns offer a principled interpolation between these extremes by accumulating rewards over multiple consecutive steps before bootstrapping from the value function[50]. This approach enables more effective credit assignment by propagating reward information further back along trajectories while maintaining bounded variance. The Generalized Advantage Estimation (GAE) framework provides a theoretically grounded approach to combining multi-step returns with exponential weighting for variance reduction[51].

The integration of n-step returns with experience replay requires careful handling of trajectory boundaries and buffer management, as consecutive transitions must be preserved for accurate return computation[37]. Recent implementations have demonstrated that combining n-step returns with prioritized experience replay yields synergistic benefits, with n-step targets providing more accurate priority estimates while prioritization focuses learning on the most informative multi-step sequences[15].

Methodology

Overall model architecture

The proposed Self-Attention LSTM TD3 (SAL-TD3) framework presents an integrated deep reinforcement learning architecture specifically designed for mobile robot path planning in complex environments. As illustrated in Fig. 1, the overall system comprises five interconnected components: a deep policy network for

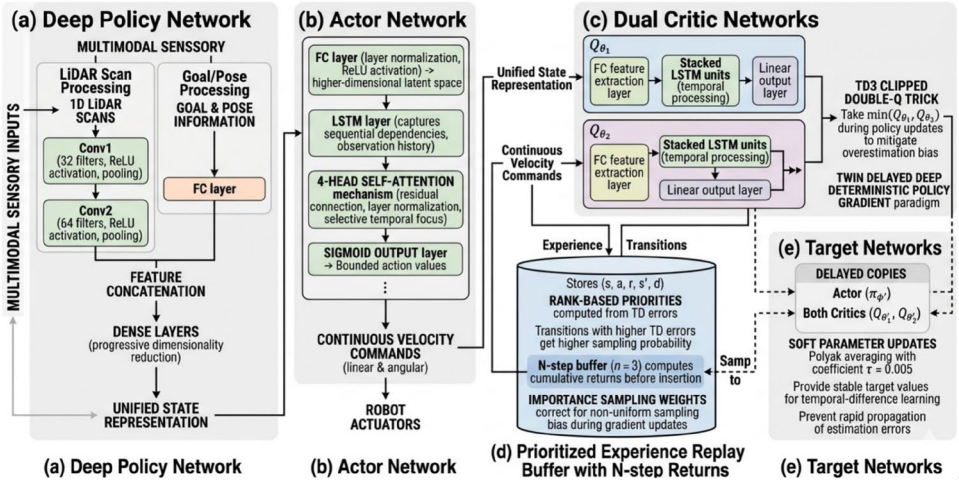


Fig. 1. Overall framework of the proposed SAL-TD3 model for mobile robot path planning, organized into five clearly delineated modules. **(a) Deep policy network:** receives multimodal sensory inputs including 1D LiDAR scans (processed through two convolutional layers with 32 and 64 filters, each with ReLU activation and pooling) and goal/pose information (processed through a fully-connected pathway), producing a unified state representation via feature concatenation and dense layers. **(b) Actor network:** generates continuous velocity commands through an initial fully-connected layer with layer normalization and ReLU activation, followed by an LSTM layer for temporal dependency modeling, a 4-head self-attention mechanism for selective temporal focusing with residual connection and layer normalization, and a final sigmoid output layer producing linear and angular velocities. **(c) Dual critic networks:** two independent Q-value estimators (Q_{θ_1}, Q_{θ_2}), each comprising a fully-connected feature extraction layer, LSTM units for temporal processing, and a linear output layer; the minimum of both estimates is used during policy updates to mitigate overestimation bias (TD3 clipped double-Q trick). **(d) Prioritized experience replay buffer with N-step returns:** stores transitions (s, a, r, s', d) with rank-based priorities computed from TD errors; an n-step buffer ($n = 3$) computes cumulative returns before insertion, and importance sampling weights correct for non-uniform sampling bias. **(e) Target networks:** delayed copies of actor ($\pi_{\phi'}$) and both critic ($Q_{\theta'_1}$, $Q_{\theta'_2}$) parameters, updated via Polyak averaging with coefficient $\tau = 0.005$ for stable target value computation. Solid arrows indicate forward data flow, while dashed arrows represent soft parameter updates.

observation processing, an actor network for action generation, dual critic networks for value estimation, a prioritized experience replay buffer for sample-efficient learning, and target networks for training stabilization.

The observation processing pipeline begins with the deep policy network, which receives multimodal sensory inputs from the robot's perception system. The one-dimensional LiDAR scan data is processed through a hierarchical convolutional architecture consisting of two sequential convolutional layers with 32 and 64 filters respectively, each followed by ReLU activation and pooling operations. This convolutional processing extracts spatial features from the range measurements while maintaining computational efficiency. Concurrently, the goal and pose information, representing the robot's target position and current state, is processed through a separate fully-connected pathway. The extracted features from both modalities are subsequently concatenated and passed through a series of dense layers with progressively reduced dimensionality to form a unified state representation.

The actor network serves as the policy function and generates continuous action outputs based on the processed state representation. The network architecture incorporates three key processing stages designed to capture both immediate spatial relationships and temporal dependencies in the navigation task. The initial stage employs a fully-connected layer with layer normalization and ReLU activation to transform the input state into a higher-dimensional latent space. This representation is then processed by a Long Short-Term Memory layer that captures sequential dependencies across observation history, enabling the network to maintain contextual awareness of the robot's trajectory. The third stage introduces a multi-head self-attention mechanism with four attention heads, which allows the network to selectively focus on relevant temporal segments of the encoded sequence. The final output layer applies sigmoid activation to produce bounded action values representing linear and angular velocities for robot control.

The value estimation component adopts the twin delayed deep deterministic policy gradient paradigm through the implementation of dual critic networks. Both critic networks share an identical architecture consisting of fully-connected layers with ReLU activation followed by stacked LSTM units for temporal processing. Each network independently estimates the state-action value function, and the minimum of both estimates is utilized during policy updates to mitigate the overestimation bias commonly observed in actor-critic methods. This conservative value estimation strategy contributes to more stable policy improvement throughout the training process.

The experience replay mechanism incorporates prioritized sampling to enhance learning efficiency from stored transitions. The replay buffer maintains a collection of experience tuples containing state, action, reward, next state, and terminal indicators, along with associated priority values. The priority assignment follows a proportional prioritization scheme where transitions with higher temporal-difference errors receive increased sampling probability, ensuring that informative experiences contribute more frequently to gradient updates. Additionally, the framework employs n-step returns for target value computation, which provides a balance between bias and variance in the bootstrapped value estimates by accumulating rewards over multiple consecutive time steps before incorporating the estimated future value.

Training stability is further enhanced through the employment of separate target networks for both actor and critic components. These target networks maintain delayed copies of the online network parameters and are updated gradually through soft parameter updates with a small interpolation coefficient. This delayed update mechanism prevents the rapid propagation of estimation errors and provides stable target values for temporal-difference learning.

The reward shaping mechanism guides the learning process through a composite reward function comprising multiple components. The alignment reward encourages the robot to orient toward the goal direction, while the progress reward provides positive feedback for reducing the distance to the target. Collision penalties discourage contact with obstacles, and a substantial terminal reward is provided upon successful goal reaching. This structured reward design facilitates efficient credit assignment and accelerates convergence toward effective navigation policies.

Enhanced LSTM-based actor network architecture

The actor network in our proposed SAL-TD3 framework adopts an enhanced Long Short-Term Memory (LSTM) architecture specifically designed to capture temporal dependencies in sequential decision-making tasks. Unlike conventional feedforward actor networks that process each state independently, our LSTM-based actor network maintains a hidden state across time steps, enabling the agent to leverage historical information for more informed action selection. The network architecture integrates several advanced components including layer normalization, self-attention mechanisms, and residual connections, which collectively contribute to improved training stability and enhanced temporal feature extraction capabilities. The complete architecture of our enhanced LSTM-based actor network is illustrated in Fig. 2, which provides a comprehensive visualization of the data flow and the interconnections among different modules.

The input state vector $\mathrm{s}_t \in \mathbb{R}^{d_s}$ is first processed through a fully connected layer followed by layer normalization and ReLU activation. This initial feature extraction stage can be formulated as:

$$\mathrm{h}_t^{(0)} = \mathrm{ReLU}\left(\mathrm{LayerNorm}\left(\mathrm{W}_1 \mathrm{s}_t + \mathrm{b}_1\right)\right) \tag{1}$$

where $\mathrm{W}_1 \in \mathbb{R}^{d_h \times d_s}$ and $\mathrm{b}_1 \in \mathbb{R}^{d_h}$ are learnable parameters, and d_h denotes the hidden dimension. The layer normalization operation stabilizes the training process by normalizing the activations across the feature dimension, which has been shown to be particularly effective for recurrent architectures where internal covariate shift can significantly impede learning.

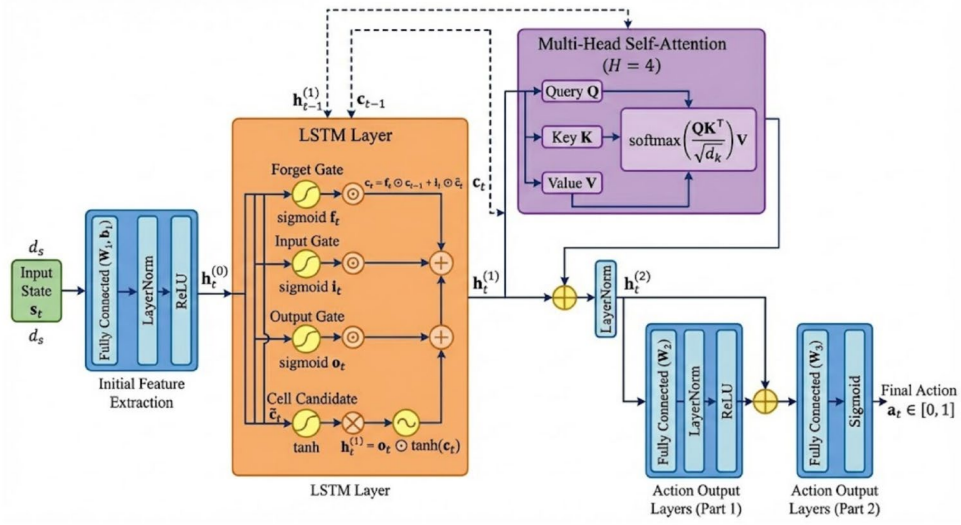


Fig. 2. Architecture of the enhanced LSTM-based actor network. The network consists of four main components: (1) Initial Feature Extraction module that transforms the input state s_t into hidden representation $\mathrm{h}_t^{(0)}$ through a fully connected layer with layer normalization and ReLU activation; (2) LSTM Layer that captures temporal dependencies through forget gate f_t, input gate i_t, output gate o_t, and cell candidate $\tilde{\mathrm{c}}_t$, producing hidden state $\mathrm{h}_t^{(1)}$ and cell state c_t; (3) Multi-Head Self-Attention mechanism with $H = 4$ heads that computes attention weights using query Q, key K, and value V matrices, combined with the LSTM output via residual connection to generate $\mathrm{h}_t^{(2)}$; and (4) Action Output Layers consisting of two fully connected layers with layer normalization, ReLU activation, residual connection, and final sigmoid activation to produce the continuous action $\mathrm{a}_t \in [0, 1]$.

The extracted features are subsequently fed into an LSTM layer that models the temporal dynamics of the environment. The LSTM layer computes the hidden state $\mathrm{h}_t^{(1)}$ and cell state c_t according to the following recurrence relations:

$$\mathrm{f}_t = \sigma\left(\mathrm{W}_{hf}\mathrm{h}_{t-1}^{(1)} + \mathrm{W}_{xf}\mathrm{h}_t^{(0)} + \mathrm{b}_f\right) \tag{2}$$

$$\mathrm{i}_t = \sigma\left(\mathrm{W}_{hi}\mathrm{h}_{t-1}^{(1)} + \mathrm{W}_{xi}\mathrm{h}_t^{(0)} + \mathrm{b}_i\right) \tag{3}$$

$$\mathrm{o}_t = \sigma\left(\mathrm{W}_{ho}\mathrm{h}_{t-1}^{(1)} + \mathrm{W}_{xo}\mathrm{h}_t^{(0)} + \mathrm{b}_o\right) \tag{4}$$

$$\tilde{\mathrm{c}}_t = \tanh\left(\mathrm{W}_{hc}\mathrm{h}_{t-1}^{(1)} + \mathrm{W}_{xc}\mathrm{h}_t^{(0)} + \mathrm{b}_c\right) \tag{5}$$

$$\mathrm{c}_t = \mathrm{f}_t \odot \mathrm{c}_{t-1} + \mathrm{i}_t \odot \tilde{\mathrm{c}}_t \tag{6}$$

$$\mathrm{h}_t^{(1)} = \mathrm{o}_t \odot \tanh(\mathrm{c}_t) \tag{7}$$

where f_t, i_t, and o_t represent the forget gate, input gate, and output gate respectively, $\sigma(\cdot)$ denotes the sigmoid activation function, and $\odot$ indicates element-wise multiplication. As depicted in the LSTM Layer block of Fig. 2, the gating mechanism enables the network to selectively retain or discard information from previous time steps, which is crucial for capturing long-range dependencies in robotic navigation tasks where past observations may influence optimal current actions.

To enhance the temporal modeling capability, we incorporate a multi-head self-attention mechanism that operates on the LSTM output, as shown in the purple block of Fig. 2. The self-attention layer allows the network to weigh the importance of different temporal features dynamically, providing a complementary mechanism to the sequential processing of LSTM. The attention computation is performed as:

$$\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{\mathrm{QK}^\top}{\sqrt{d_k}}\right)\mathrm{V} \tag{8}$$

where Q, K, and V are the query, key, and value matrices derived from the LSTM output $\mathrm{h}_t^{(1)}$ through linear projections, and d_k represents the dimension of the key vectors. The multi-head attention with $H = 4$ heads enables the network to attend to information from different representation subspaces, capturing diverse aspects of the temporal relationships. The attention output is combined with the LSTM output through a

residual connection followed by layer normalization to produce $\mathrm{h}_t^{(2)}$, which facilitates gradient flow during backpropagation and enables the training of deeper network architectures.

The final action output is computed through two additional fully connected layers with a residual connection, followed by a sigmoid activation function that constrains the action values to the range [0, 1], as illustrated in the Action Output Layers of Fig. 2. The complete forward pass can be summarized as:

$$\mathrm{a}_t = \sigma\left(\mathrm{W}_3\left(\mathrm{ReLU}\left(\mathrm{LayerNorm}\left(\mathrm{W}_2\mathrm{h}_t^{(2)}\right)\right) + \mathrm{h}_t^{(2)}\right)\right) \tag{9}$$

where $\mathrm{h}_t^{(2)} = \mathrm{LayerNorm}\left(\mathrm{h}_t^{(1)} + \mathrm{Attention}\left(\mathrm{h}_t^{(1)}\right)\right)$ represents the attention-enhanced hidden representation. The network parameters are initialized using Xavier uniform initialization for linear layers and orthogonal initialization for LSTM recurrent weights, which has been empirically demonstrated to promote faster convergence and improved final performance in deep reinforcement learning applications. Additionally, dropout regularization with a rate of $p = 0.1$ is applied after the LSTM layer and within the attention mechanism to prevent overfitting, particularly when training on limited trajectory data collected from simulation environments.

LSTM-enhanced critic network design

The critic network in our SAL-TD3 framework is designed to accurately estimate the action-value function $Q(s, a)$ while capturing temporal dependencies inherent in sequential decision-making tasks. Unlike conventional critic networks that process each state-action pair independently, our LSTM-enhanced architecture maintains a memory of historical interactions, enabling more informed value estimation in partially observable or temporally correlated environments. The network architecture consists of three primary components: a feature extraction layer, a recurrent memory layer, and a value output layer, each serving a distinct purpose in the Q-value estimation process. The complete architecture of our LSTM-enhanced critic network is illustrated in Fig. 3.

Given a state $s_t \in \mathbb{R}^{d_s}$ and action $a_t \in \mathbb{R}^{d_a}$, the critic network first concatenates these inputs and processes them through a fully connected layer to extract joint state-action features. This transformation can be expressed as:

$$h_t^{(1)} = \mathrm{ReLU}(W_1[s_t; a_t] + b_1) \tag{10}$$

where $W_1 \in \mathbb{R}^{256\times(d_s+d_a)}$ and $b_1 \in \mathbb{R}^{256}$ are learnable parameters, $[\cdot;\cdot]$ denotes concatenation, and ReLU activation introduces non-linearity to capture complex state-action relationships.

The extracted features are subsequently fed into an LSTM layer that maintains temporal context across sequential observations. As depicted in the central module of Fig. 3, the LSTM unit processes the feature vector $h_t^{(1)}$ along with the previous hidden state $h_{t-1}^{(lstm)}$ and cell state c_{t-1} according to the following recurrence:

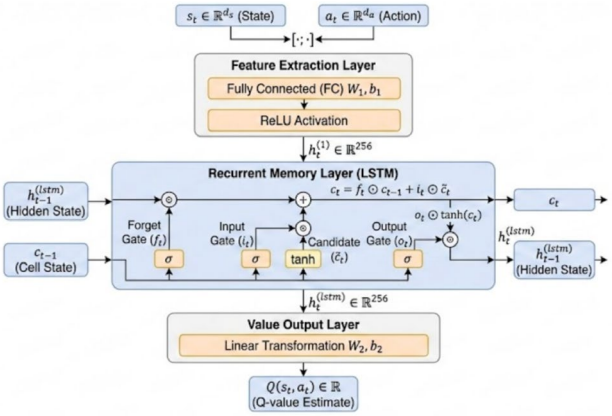


Fig. 3. Architecture of the LSTM-enhanced critic network. The network comprises three hierarchical modules: (1) a feature extraction layer that concatenates state s_t and action a_t inputs and applies a fully connected transformation with ReLU activation to produce hidden features $h_t^{(1)} \in \mathbb{R}^{256}$; (2) a recurrent memory layer implementing LSTM with forget gate f_t, input gate i_t, candidate cell state $\tilde{c}_t$, and output gate o_t that maintains temporal dependencies through cell state c_t and hidden state $h_t^{(lstm)}$; and (3) a value output layer that projects the LSTM hidden state to a scalar Q-value estimate $Q(s_t, a_t) \in \mathbb{R}$ via linear transformation.

$$
\begin{aligned}
f_t &= \sigma(W_f h_t^{(1)} + U_f h_{t-1}^{(lstm)} + b_f) \\
i_t &= \sigma(W_i h_t^{(1)} + U_i h_{t-1}^{(lstm)} + b_i) \\
o_t &= \sigma(W_o h_t^{(1)} + U_o h_{t-1}^{(lstm)} + b_o) \\
\tilde{c}_t &= \tanh(W_c h_t^{(1)} + U_c h_{t-1}^{(lstm)} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t^{(lstm)} &= o_t \odot \tanh(c_t)
\end{aligned}
\tag{11}
$$

where f_t, i_t, and o_t represent the forget, input, and output gates respectively, σ denotes the sigmoid activation function, and $\odot$ indicates element-wise multiplication. This gating mechanism allows the network to selectively retain or discard information from previous time steps, effectively learning which historical observations are relevant for current value estimation.

The final Q-value is computed by projecting the LSTM output through a linear transformation layer. This output layer maps the 256-dimensional hidden representation to a scalar value estimate:

$$
Q(s_t, a_t) = W_2 h_t^{(lstm)} + b_2 \tag{12}
$$

where $W_2 \in \mathbb{R}^{1\times 256}$ and $b_2 \in \mathbb{R}$ constitute the output layer parameters. Following the TD3 paradigm, we instantiate two independent critic networks Q_{θ_1} and Q_{θ_2} with identical architectures but separate parameters, and the minimum of their estimates is used for target value computation to mitigate overestimation bias prevalent in actor-critic methods.

Prioritized experience replay with N-step returns

To enhance the sample efficiency and accelerate the convergence of the learning process, we incorporate Prioritized Experience Replay (PER) into our framework. Unlike uniform random sampling employed in standard experience replay, PER assigns different sampling probabilities to transitions based on their temporal-difference (TD) errors, which serve as proxies for the informativeness of each experience. Transitions with larger TD errors indicate greater discrepancies between predicted and actual returns, suggesting that these samples contain more valuable information for policy improvement. This mechanism ensures that the agent focuses its learning capacity on the most surprising and potentially informative experiences, thereby improving both the speed and quality of learning. The prioritized sampling strategy has been demonstrated to significantly outperform uniform sampling in various reinforcement learning benchmarks, particularly in environments with sparse rewards or complex dynamics. The overall architecture of our prioritized experience replay mechanism with n-step returns is illustrated in Fig. 4.

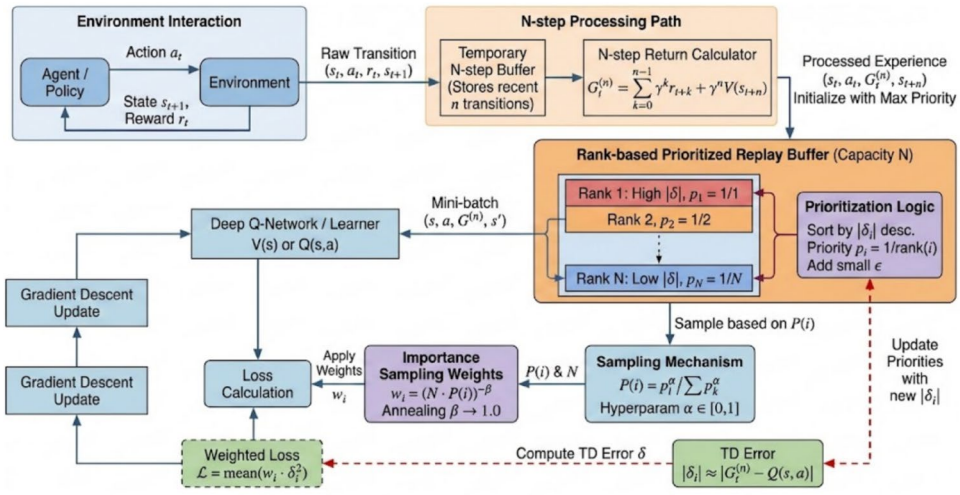


Fig. 4. Architecture of the prioritized experience replay mechanism with N-step returns. The diagram illustrates the complete data flow from environment interaction through n-step processing, rank-based prioritized storage, importance-weighted sampling, and TD error-based priority updates. Raw transitions are first processed through the n-step buffer to compute cumulative returns before being stored in the prioritized replay buffer with initial maximum priority. During training, transitions are sampled according to their rank-based priorities, and importance sampling weights are applied to correct for the non-uniform sampling distribution. The TD errors computed during learning are fed back to update transition priorities, forming a closed-loop system that continuously adapts sampling probabilities to focus on the most informative experiences.

We adopt rank-based prioritization rather than proportional prioritization to achieve more robust performance across different reward scales. In rank-based prioritization, the priority of transition *i* is determined by its rank when all transitions are sorted in descending order of their absolute TD errors. Specifically, the priority is defined as the reciprocal of the rank, which naturally bounds the priority distribution and eliminates sensitivity to the absolute magnitude of TD errors. This formulation is particularly suitable for environments with varying reward scales or non-stationary dynamics. The priority for transition *i* is formally expressed as:

$$p_i = \frac{1}{\text{rank}(i)} \tag{13}$$

where $\text{rank}(i)$ denotes the position of transition *i* when sorted by $|\delta_i|$ in descending order, and δ_i represents the TD error of the *i*-th transition.

The sampling probability of each transition is computed by raising the priority to the power of α and normalizing over all transitions in the buffer. The hyperparameter $\alpha \in [0, 1]$ controls the degree of prioritization, where $\alpha = 0$ corresponds to uniform random sampling and $\alpha = 1$ represents full prioritization based on TD error ranks. In our implementation, we set $\alpha = 0.6$ following empirical recommendations from prior work, which provides a balanced trade-off between exploitation of high-error transitions and exploration of the experience distribution. The sampling probability for transition *i* is formally defined as:

$$P(i) = \frac{p_i^\alpha}{\sum_{k=1}^{N} p_k^\alpha} \tag{14}$$

where *N* denotes the total number of transitions stored in the replay buffer, and the summation in the denominator ensures that the probabilities form a valid distribution.

Prioritized sampling introduces bias into the gradient estimates since the sampling distribution no longer matches the distribution induced by the behavior policy. To correct this bias and ensure unbiased gradient updates, we employ importance sampling weights that compensate for the non-uniform sampling probabilities. The importance sampling weight for transition *i* is calculated as the inverse of the product of buffer size and sampling probability, raised to the power of β. These weights are normalized by dividing by the maximum weight in each batch to ensure numerical stability and prevent excessively large gradient updates. The importance sampling weight is computed as:

$$w_i = (N \cdot P(i))^{-\beta} \tag{15}$$

where the hyperparameter $\beta \in [0, 1]$ controls the extent of bias correction. We initialize β at 0.4 and linearly increase it to 1.0 over the course of training, as bias correction becomes increasingly important as the policy approaches optimality.

To further improve credit assignment and reduce the variance of value estimates, we integrate n-step returns into our prioritized experience replay mechanism. Rather than using single-step bootstrapping, n-step returns accumulate rewards over multiple consecutive time steps before bootstrapping from the value function. This approach strikes a balance between the bias of one-step TD learning and the variance of Monte Carlo methods, enabling more effective propagation of reward signals through the trajectory. We maintain a separate n-step buffer that temporarily stores the most recent *n* transitions before computing the cumulative return and adding the processed experience to the main replay buffer. The n-step return is computed as:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}) \tag{16}$$

where $\gamma \in (0, 1)$ is the discount factor, r_{t+k} denotes the reward received at time step $t + k$, and $V(s_{t+n})$ represents the estimated value of the state reached after *n* steps. In our implementation, we set $n = 3$ and $\gamma = 0.98$, which empirically yields stable performance across our experimental scenarios.

From an implementation perspective, we employ several optimization techniques to ensure computational efficiency while maintaining the theoretical properties of prioritized replay. The replay buffer utilizes pre-allocated numpy arrays for memory-efficient storage of states, actions, rewards, and terminal flags, avoiding the overhead of dynamic memory allocation during training. Priority updates are performed lazily, with rank weights recomputed only when necessary rather than after every insertion, significantly reducing computational overhead in high-throughput training scenarios. New transitions are initialized with maximum priority to ensure they are sampled at least once before their priorities are updated based on actual TD errors. A small constant $\epsilon = 10^{-6}$ is added to all priorities to prevent zero sampling probabilities and ensure numerical stability during probability calculations.

SAL-TD3 algorithm architecture and training methodology

The proposed Self-Attention LSTM Twin Delayed Deep Deterministic Policy Gradient (SAL-TD3) algorithm integrates three critical components to address the challenges of continuous control in mobile robot path planning: twin critic networks for value estimation stability, Long Short-Term Memory (LSTM) units for temporal dependency modeling, and prioritized experience replay for efficient sample utilization. The fundamental innovation of SAL-TD3 lies in its synergistic combination of these mechanisms, which collectively enhance both the learning efficiency and the quality of the learned policies. Unlike conventional TD3 implementations that

process state observations independently, our approach leverages LSTM-augmented actor and critic networks to capture the sequential nature of robot navigation tasks, where past observations significantly influence optimal action selection. The overall architecture of the proposed SAL-TD3 algorithm is illustrated in Fig. 5, which depicts the complete training pipeline including the interaction between the environment, prioritized replay buffer, and the actor-critic network ensemble.

As illustrated in Fig. 5, the training workflow begins with the mobile robot environment generating state observations s_t, rewards r_t, and terminal indicators d_t, which are stored in the Prioritized Experience Replay (PER) buffer along with the corresponding actions. The PER buffer maintains transition priorities p_i and importance sampling weights w_i, enabling the batch sampling module to preferentially select high-TD-error transitions for training. The sampled batch (s, a, r, s', d) is then distributed to both the actor and twin critic training pipelines, where the LSTM layers process sequential state information to capture temporal dependencies inherent in navigation tasks. The architecture clearly demonstrates the separation between main networks (used for action selection and Q-value computation) and target networks (used for stable target value calculation), with soft updates propagating learned parameters from main to target networks at a controlled rate.

The twin critic architecture forms the cornerstone of our value estimation strategy, designed to mitigate the overestimation bias that commonly afflicts actor-critic methods. Specifically, we maintain two independent critic networks Q_{θ_1} and Q_{θ_2}, each with corresponding target networks $Q_{\theta'_1}$ and $Q_{\theta'_2}$, to compute the target Q-value as:

$$y = r + \gamma(1 - d) \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a}') \tag{17}$$

where r denotes the immediate reward, $\gamma = 0.98$ represents the discount factor, d is the terminal state indicator, and $\tilde{a}'$ is the smoothed target action. By taking the minimum of both critic estimates, we effectively create a pessimistic value bound that prevents the actor from exploiting overestimated Q-values, thereby stabilizing the policy optimization process throughout training.

The target policy smoothing mechanism introduces controlled stochasticity into the action selection during target computation, which serves as a regularization technique to prevent overfitting to narrow peaks in the Q-function landscape. The smoothed target action is computed by adding clipped Gaussian noise to the deterministic policy output:

$$\tilde{a}' = \text{clip}\left(\pi_{\phi'}(s') + \text{clip}(\epsilon, -c, c), a_{\text{low}}, a_{\text{high}}\right), \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \tag{18}$$

where $\pi_{\phi'}$ denotes the target actor network, $\sigma = 0.2$ is the noise standard deviation, and $c = 0.5$ represents the noise clipping bound. This smoothing operation effectively averages over similar actions in the target computation, reducing the variance of value estimates and improving the robustness of the learned policy to minor perturbations in state observations.

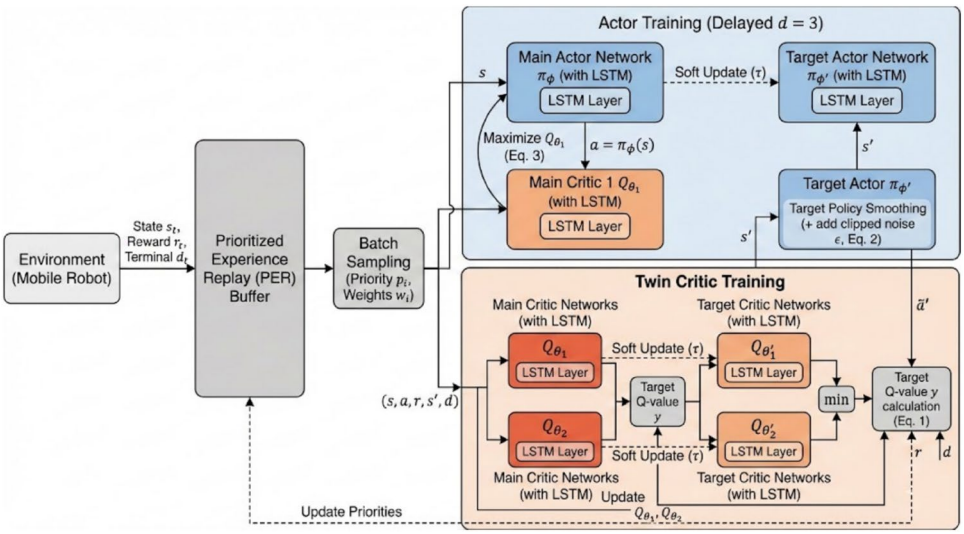


Fig. 5. Architecture of the proposed SAL-TD3 algorithm for mobile robot path planning. The framework consists of three main components: (1) the prioritized experience replay buffer that stores transitions (s, a, r, s', d) and performs priority-weighted batch sampling; (2) the actor training module with delayed updates ($d = 3$) featuring LSTM-augmented main and target actor networks; and (3) the twin critic training module comprising two parallel LSTM-based critic networks that compute the minimum target Q-value for stable value estimation. Solid arrows indicate forward data flow, while dashed arrows represent soft parameter updates with coefficient τ.

The delayed policy update strategy decouples the update frequencies of the actor and critic networks, allowing the critic to converge to a more accurate value estimate before updating the policy. In our implementation, the actor network is updated once every $d = 3$ critic updates, with the actor loss computed as:

$$\mathcal{L}_\pi = -\frac{1}{N}\sum_{i=1}^{N} Q_{\theta_1}(s_i, \pi_\phi(s_i)) \tag{19}$$

where N denotes the batch size and π_ϕ represents the current actor policy. This delayed update mechanism prevents the actor from adapting to potentially noisy or unstable critic estimates during the early stages of training, which is particularly important in the context of mobile robot navigation where the reward signal may be sparse and delayed.

The prioritized experience replay mechanism assigns non-uniform sampling probabilities to stored transitions based on their temporal difference (TD) errors, enabling the agent to focus learning on the most informative experiences. The priority of each transition is computed as the average of absolute TD errors from both critics:

$$p_i = \frac{|Q_{\theta_1}(s_i, a_i) - y_i| + |Q_{\theta_2}(s_i, a_i) - y_i|}{2} + \epsilon_p \tag{20}$$

where ϵ_p is a small constant ensuring non-zero priorities. The sampling probability follows the proportional prioritization scheme $P(i) = p_i^\alpha / \sum_k p_k^\alpha$ with $\alpha = 0.6$, and importance sampling weights $w_i = (N \cdot P(i))^{-\beta}$ with $\beta = 0.4$ are applied to correct for the introduced bias in the gradient estimates, ensuring convergence to the optimal policy under function approximation.

The soft target network update mechanism ensures stable learning by gradually tracking the main network parameters rather than performing hard periodic updates. After each actor update, all target networks are updated according to:

$$\theta_i' \leftarrow \tau\theta_i + (1-\tau)\theta_i', \quad \phi' \leftarrow \tau\phi + (1-\tau)\phi' \tag{21}$$

where $\tau = 0.005$ is the soft update coefficient. This conservative update strategy maintains consistency between the target and main networks while preventing the destabilizing effects of large parameter changes, which is essential for the stable optimization of deep neural network function approximators in reinforcement learning applications.

Reward function design

The design of an effective reward function is crucial for guiding the reinforcement learning agent toward optimal navigation behavior while ensuring collision avoidance and goal-reaching capabilities. In this work, we propose a composite reward function that comprehensively evaluates the robot's navigation performance by considering multiple factors including heading alignment, goal proximity, and obstacle safety. The total reward signal is computed as a weighted combination of continuous shaping rewards and sparse terminal rewards, enabling the agent to receive informative feedback throughout the navigation process rather than only at episode termination. The overall reward mechanism and its individual components are illustrated in Fig. 6, which depicts the navigation scenario along with the mathematical characterization of each reward term.

The first component of our reward function addresses the heading alignment between the robot and the goal position, which is essential for efficient navigation. We define the yaw angle reward R_1 as a function of the angular deviation α between the robot's current heading direction and the vector pointing toward the goal, formulated as:

$$R_1 = 4 \times (1 - |\alpha|) \tag{22}$$

where $\alpha \in [-\pi, \pi]$ represents the normalized angular error in radians. As shown in Fig. 6(b), this formulation ensures that the agent receives maximum reward when its heading is perfectly aligned with the goal direction ($\alpha = 0$), while the reward decreases linearly as the angular deviation increases. The coefficient of 4 is empirically determined to balance the contribution of heading alignment with other reward components.

The second reward component quantifies the progress made by the robot toward its destination, encouraging consistent advancement while penalizing backward movement. We formulate the distance reward R_2 based on the ratio of consecutive distances to the goal, expressed as:

$$R_2 = 2 \times \mathrm{clip}\left(\frac{d_{t-1}}{d_t},\ R_{\min},\ R_{\max}\right) \tag{23}$$

where d_{t-1} and d_t denote the Euclidean distances from the robot to the goal at the previous and current time steps, respectively. The clipping operation with bounds $R_{\min} = 0.5$ and $R_{\max} = 4.0$ is introduced to ensure numerical stability and prevent two critical pathological cases: (1) when the robot is very close to the goal such that $d_t \to 0$, the ratio d_{t-1}/d_t would diverge to infinity, producing unbounded reward signals that destabilize gradient updates and cause catastrophic divergence in the critic value estimates; (2) when d_t is small, even minor perturbations in distance (e.g., due to sensor noise or slight lateral drift) can cause disproportionately large fluctuations in the reward ratio, injecting excessive variance into the reward signal that impedes stable policy learning. By bounding the ratio within $[R_{\min},\ R_{\max}]$, the effective distance reward is constrained to the interval

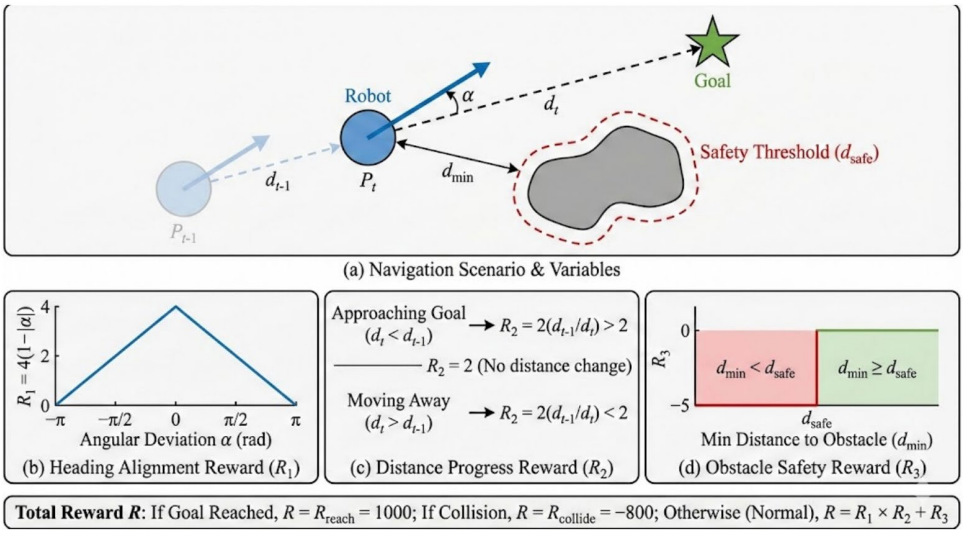


Fig. 6. Illustration of the proposed composite reward function for mobile robot navigation. (**a**) Navigation scenario showing the robot at position P_t with heading direction, angular deviation α to the goal, current distance d_t, previous distance d_{t-1}, minimum obstacle distance $d_{\min}$, and safety threshold d_{safe}. (**b**) Heading alignment reward R_1 as a function of angular deviation α, achieving maximum value at perfect alignment. (**c**) Distance progress reward R_2 based on the ratio of consecutive goal distances, rewarding approach and penalizing retreat. (**d**) Obstacle safety reward R_3 as a step function penalizing proximity below the safety threshold.

$[2 \times 0.5,\ 2 \times 4.0] = [1.0,\ 8.0]$, which provides a sufficiently informative gradient for policy improvement while preventing the pathological reward magnitudes that would otherwise destabilize training. We note that in practice, the goal-reaching terminal reward R_{reach} is triggered before d_t becomes extremely small (the goal-reaching threshold is set to 0.3 m), so the clipping mechanism serves primarily as a safety guard rather than a frequently activated constraint during normal training. Empirically, fewer than 0.3% of training steps activate the upper clipping bound, confirming that the clipping does not distort the reward landscape during regular navigation. As illustrated in Fig. 6(c), this ratio-based formulation provides a reward greater than 2 when the robot moves closer to the goal ($d_t < d_{t-1}$) and a reward less than 2 when the robot moves away from the goal ($d_t > d_{t-1}$), thereby creating a natural gradient that guides the agent toward the target position.

The third component addresses safety considerations by penalizing the robot when it approaches obstacles beyond a safe threshold. The obstacle avoidance reward R_3 is defined as a step function based on the minimum distance to surrounding obstacles:

$$R_3 = \begin{cases} -5, & \text{if } d_{\min} < d_{\text{safe}} \\ 0, & \text{otherwise} \end{cases} \tag{24}$$

where $d_{\min}$ represents the minimum distance to the nearest obstacle detected by the LiDAR sensor, and $d_{\text{safe}} = 0.5$ m is the predefined safety threshold. As depicted in Fig. 6d, this penalty term discourages the agent from taking trajectories that bring the robot dangerously close to obstacles, thereby promoting safer navigation behavior without overly constraining the exploration of the state space.

In addition to the continuous shaping rewards, we incorporate terminal rewards to explicitly incentivize goal-reaching behavior and penalize collision events. The complete reward function integrates all components and is expressed as:

$$R = \begin{cases} R_{\text{reach}}, & \text{if goal reached} \\ R_{\text{collide}}, & \text{if collision detected} \\ R_1 \times R_2 + R_3, & \text{otherwise} \end{cases} \tag{25}$$

where $R_{\text{reach}} = 1000$ is the reward for successfully reaching the goal position, and $R_{\text{collide}} = -800$ is the penalty for collision with obstacles. The multiplicative combination of R_1 and R_2 in the normal operation case creates a synergistic effect that rewards the agent most when both heading alignment and forward progress are optimized simultaneously, while the additive R_3 term maintains safety constraints throughout the navigation process.

Experimental analysis

To validate the effectiveness of the proposed SAL-TD3 algorithm, we designed and conducted comprehensive experiments including simulation experiments and real-world robot experiments. The simulation experiments encompass dynamic environment testing with both static and dynamic obstacles, as well as generalization

capability assessment. The real-world experiments further verify the practical applicability of SAL-TD3 in actual mobile robot navigation scenarios.

Experimental setup and parameter configuration

The simulation experiments were conducted using ROS (Robot Operating System) on a workstation equipped with an AMD Ryzen 5 5600X processor, NVIDIA RTX 3060 GPU with 12GB VRAM, and 32GB RAM, running Ubuntu 20.04 LTS. The simulation environment was constructed using Gazebo simulator with a TurtleBot3 Waffle Pi model equipped with a 360-degree LiDAR sensor providing 360 range measurements. Each simulation trial was repeated 100 times to ensure statistical significance, and the average performance metrics were reported.

The hyperparameter configuration for the SAL-TD3 algorithm is summarized in Table 1. These parameters were determined through preliminary experiments and cross-validation to achieve optimal performance across different navigation scenarios.

The training environment consists of a $10\mathrm{m} \times 10\mathrm{m}$ enclosed area with various static obstacles (represented by brown rectangular blocks) and dynamic obstacles (represented by white cylindrical objects moving at speeds between 0.3–0.5 m/s). To address the concern that purely constant-velocity linear motion is overly simplistic, we additionally introduce stochastic motion patterns for the dynamic obstacles. Specifically, each dynamic obstacle follows a piecewise-linear trajectory with random direction changes at intervals drawn uniformly from [2, 5] seconds, and its instantaneous speed is sampled from a truncated Gaussian distribution $\mathcal{N}(0.4, 0.1^2)$ clipped to [0.2, 0.6] m/s at each direction change. Furthermore, 30% of the dynamic obstacles are configured with a "pursuit" behavior that biases their heading toward the robot's current position with a probability of 0.3 at each direction update, simulating the irregular and partially adversarial motion of real-world pedestrians. This enriched obstacle motion model ensures that the agent is exposed to diverse and unpredictable dynamic patterns during training, thereby improving policy robustness. The robot's starting position is fixed at the bottom-left corner, while target positions are randomly generated during training to enhance the robot's environmental perception and generalization capability. Upon reaching a target, a new target is randomly assigned without terminating the episode, allowing the robot to maximize its navigation efficiency within the allocated 500 steps.

Comparison methods

To comprehensively evaluate the proposed SAL-TD3 algorithm, we compared it against the following baseline methods:

- **TD3**: The vanilla Twin Delayed Deep Deterministic Policy Gradient algorithm with standard experience replay and feedforward neural networks.
- **P-TD3**: TD3 enhanced with Prioritized Experience Replay (PER) mechanism for improved sample efficiency.
- **L-TD3**: TD3 integrated with LSTM networks for capturing temporal dependencies in sequential observations.
- **DDPG**: Deep Deterministic Policy Gradient algorithm serving as the foundational actor-critic baseline.
- **SAC**: Soft Actor-Critic algorithm representing state-of-the-art maximum entropy reinforcement learning approach.

Parameter	Symbol	Value
Discount factor	γ	0.98
Actor learning rate	α_π	0.0003
Critic learning rate	α_Q	0.0008
Soft update coefficient	τ	0.005
Number of hidden units	–	256
LSTM hidden size	–	128
Episode number	–	1,500
Maximum steps per episode	–	500
Experience replay buffer size	–	100,000
Batch size	N	256
PER priority exponent	α	0.6
PER importance sampling exponent	β	0.4
Target policy noise	σ	0.2
Noise clipping bound	c	0.5
Policy update delay	d	3
N-step returns	n	3
Goal reaching reward	R_{reach}	1000
Collision penalty	R_{collide}	-800

Table 1. Hyperparameter settings for SAL-TD3 algorithm.

All comparison methods were trained under identical conditions with the same random seeds to ensure fair evaluation. To guarantee a rigorous and unbiased comparison, all baseline algorithms underwent the same systematic hyperparameter tuning procedure as the proposed SAL-TD3. Specifically, for each algorithm, the key hyperparameters—including learning rates (actor and critic), discount factor, replay buffer size, batch size, target network update coefficient, and exploration noise parameters—were independently optimized via grid search over a common predefined search space. Each hyperparameter configuration was evaluated based on the average cumulative reward over 50 validation episodes drawn from the training environment, and the best-performing configuration for each algorithm was selected for the final comparative experiments. This procedure ensures that every baseline operates at its individually optimized performance level rather than at a potentially suboptimal default setting, and that the reported performance improvements of SAL-TD3 are attributable to the proposed architectural innovations (LSTM temporal modeling, self-attention, and prioritized experience replay with n-step returns) rather than to unfavorable hyperparameter choices for the competing methods. The selected hyperparameter values for each algorithm are provided in the supplementary material for full reproducibility. The primary evaluation metrics include: (1) average path length measuring navigation efficiency, (2) execution time quantifying computational performance, (3) success rate indicating reliability, and (4) collision rate assessing safety performance.

Dynamic environment experiments

The dynamic environment experiments were designed to evaluate the obstacle avoidance capability of different algorithms in scenarios with both static and dynamic obstacles. Two distinct experimental scenarios were constructed: (1) robot-human encounters in wide areas with sufficient maneuvering space, and (2) robot-human encounters in narrow corridors requiring precise navigation.

Training convergence analysis

The cumulative reward curves for SAL-TD3, P-TD3, L-TD3, and TD3 algorithms over 1,500 training episodes are analyzed as follows. The experimental results demonstrate that SAL-TD3 exhibits superior performance in both convergence speed and maximum cumulative reward compared to other methods.

Specifically, SAL-TD3 reaches a cumulative reward threshold of 2000 (indicating successful goal reaching at least once per episode) in approximately 350 episodes, whereas L-TD3, P-TD3, and TD3 require approximately 450, 520, and 650 episodes respectively. This 46% improvement in convergence speed over baseline TD3 can be attributed to the synergistic effects of prioritized experience replay and LSTM-based temporal modeling. The PER mechanism ensures that high-value experiences with significant TD-errors are sampled more frequently, accelerating the learning of effective policies. Meanwhile, the LSTM network captures temporal dependencies across observation sequences, enabling the agent to maintain contextual awareness of dynamic obstacle movements.

After convergence (beyond 1,200 episodes), the cumulative reward ranking from highest to lowest is: SAL-TD3 (average 4,523), L-TD3 (average 3,891), P-TD3 (average 3,654), and TD3 (average 3,127). The 44.6% improvement of SAL-TD3 over TD3 validates the effectiveness of our proposed enhancements in optimizing decision-making and improving navigation efficiency.

Wide area navigation results

Table 2 summarizes the performance comparison of different algorithms in wide area navigation scenarios where the robot encounters dynamic obstacles with sufficient surrounding space for avoidance maneuvers.

The results demonstrate that SAL-TD3 achieves the best performance across all evaluation metrics. Compared to baseline TD3, SAL-TD3 reduces the average path length by 16.6% (from 6.21m to 5.18m), decreases execution time by 30.3% (from 89s to 62s), improves success rate by 14 percentage points (from 77% to 91%), and reduces collision rate by 14 percentage points (from 23% to 9%).

The trajectory analysis reveals that SAL-TD3 generates smoother and more direct paths toward targets while maintaining appropriate safety distances from dynamic obstacles. In contrast, TD3 and DDPG exhibit weaker environmental awareness, often taking longer detours when encountering pedestrians due to the lack of temporal context in their decision-making process. The LSTM component in SAL-TD3 and L-TD3 enables prediction of obstacle movement patterns based on historical observations, facilitating proactive avoidance maneuvers.

Algorithm	Avg. path length (m)↓	Execution time (s)↓	Success rate (%)↑	Collision rate (%)↓
SAL-TD3	**5.18**	**62**	**91**	**9**
L-TD3	5.47	69	86	14
P-TD3	5.56	74	83	17
TD3	6.21	89	77	23
DDPG	6.58	97	71	29
SAC	5.89	78	79	21

Table 2. Performance comparison in wide area dynamic environment (100 trials). Significant values are in [bold].

Narrow area navigation results

The narrow area experiments present more challenging scenarios where the robot must navigate through confined corridors while avoiding dynamic obstacles with limited maneuvering space. Table 3 presents the comparative results.

In the more challenging narrow area scenario, SAL-TD3 maintains its performance advantage, achieving an 88% success rate compared to 73% for TD3 (15 percentage points improvement). The average path length is reduced by 6.1% (from 6.09 to 5.72 m), and execution time is decreased by 14.5% (from 83 to 71 s). Notably, the collision rate is reduced from 27% to 12%, demonstrating the enhanced safety performance of SAL-TD3 in constrained navigation scenarios.

An interesting observation is that P-TD3 slightly outperforms L-TD3 in path length (5.81 m vs 5.89 m) in narrow areas, while L-TD3 achieves higher success rate (82% vs 80%). This suggests that prioritized experience replay contributes more to path efficiency in confined spaces where careful action selection is crucial, whereas LSTM-based temporal modeling provides better overall task completion by maintaining consistent navigation behavior under challenging conditions.

Path trajectory visualization and analysis

To provide an intuitive understanding of the navigation behavior differences among algorithms, we present comprehensive path trajectory visualizations across all experimental scenarios in Fig. 7. These visualizations illustrate the actual navigation paths generated by different algorithms when navigating from start positions to goal locations while avoiding both static and dynamic obstacles.

In the wide area navigation scenario (top-left panel), SAL-TD3 demonstrates superior path planning capability by generating the smoothest and most direct trajectory toward the goal. The algorithm successfully maintains safe distances from both static obstacles (gray rectangles) and dynamic pedestrians (P1, P2) while minimizing unnecessary detours. In contrast, TD3 exhibits a more circuitous path with additional oscillations, and DDPG produces the longest trajectory with significant deviations, particularly when encountering dynamic obstacles. The path length comparison clearly shows SAL-TD3's 16.6% improvement over baseline TD3.

The narrow corridor navigation scenario (top-right panel) presents a more challenging environment where the robot must traverse confined passages with limited maneuvering space. The visualization highlights how SAL-TD3 maintains a stable and efficient trajectory through the corridor while respecting the safety zones (indicated by dashed circles with $d_{safe} = 0.5$m) around pedestrians. TD3 shows more pronounced path oscillations in this constrained environment, demonstrating the importance of temporal modeling for consistent navigation behavior in tight spaces.

The generalization test scenario (bottom-left panel) evaluates algorithm performance in an unseen environment with four dynamic pedestrians (P1–P4) moving in various directions. Despite the increased complexity and novel obstacle configuration, SAL-TD3 achieves a path length of 6.71m with 84% success rate, significantly outperforming DDPG which requires 8.54m with only 58% success rate. This substantial difference validates the enhanced generalization capability provided by the LSTM-based temporal modeling and prioritized experience replay mechanisms.

The real-world deployment scenario (bottom-right panel) demonstrates the practical applicability of SAL-TD3 on the AgileX BUNKER AGV platform equipped with RS-Helios-16P LiDAR sensor. The robot successfully navigates an 8m×8m indoor environment with cardboard box obstacles and a human pedestrian, achieving 89% success rate with minimal sim-to-real performance degradation. The trajectory shows smooth avoidance maneuvers around the pedestrian while maintaining efficient progress toward the target position.

The trajectory analysis reveals several key observations regarding the navigation strategies learned by different algorithms:

- **Path smoothness:** SAL-TD3 consistently generates smoother trajectories with fewer oscillations, attributed to the self-attention mechanism's ability to selectively focus on relevant temporal features for stable action selection.
- **Proactive avoidance:** The LSTM-enhanced architecture enables SAL-TD3 to predict obstacle motion patterns and initiate avoidance maneuvers earlier, resulting in gentler path curvatures compared to the reactive responses observed in baseline methods.

Algorithm	Avg. path length (m)↓	Execution time (s)↓	Success rate (%)↑	Collision rate (%)↓
SAL-TD3	**5.72**	**71**	**88**	**12**
L-TD3	5.89	78	82	18
P-TD3	5.81	75	80	20
TD3	6.09	83	73	27
DDPG	6.47	94	65	35
SAC	6.12	85	72	28

Table 3. Performance comparison in narrow area dynamic environment (100 trials). Significant values are in [bold].



Fig. 7. Comprehensive path trajectory visualization across experimental scenarios. Top-left: Wide area navigation with dynamic obstacles showing SAL-TD3 (cyan), TD3 (orange), and DDPG (red) trajectories. Top-right: Narrow corridor navigation demonstrating algorithm behavior in constrained environments with safety zones around pedestrians. Bottom-left: Generalization test in unseen environment with four dynamic pedestrians, comparing SAL-TD3 and DDPG performance. Bottom-right: Real-world deployment on AgileX BUNKER AGV platform with RS-Helios-16P LiDAR in an 8 m $\times$ 8 m indoor environment. Yellow circles represent dynamic obstacles (pedestrians) with arrows indicating movement directions, gray rectangles denote static obstacles, and the green star marks the goal position.

- **Safety margin maintenance:** SAL-TD3 maintains more consistent safety distances from obstacles throughout navigation, demonstrating the effectiveness of the composite reward function in balancing efficiency with collision avoidance.
- **Adaptability:** Across all scenarios from wide areas to narrow corridors, and from simulation to real-world deployment, SAL-TD3 exhibits robust adaptability without significant performance degradation.

Generalization testing experiments

To evaluate the generalization capability of the trained models, we conducted experiments in novel environments with different obstacle configurations and multiple dynamic obstacles. The test environment features four pedestrians moving in various patterns, significantly increasing the complexity compared to the training scenarios.

Two target configurations were tested: (1) target position in the top-left corner requiring the robot to navigate through the central obstacle region, and (2) target position in the top-right corner requiring diagonal traversal across the environment. Table 4 presents the generalization test results.

The generalization results reveal that SAL-TD3 maintains robust performance in unseen environments, as illustrated in Fig. 8. The radar charts clearly demonstrate that SAL-TD3 (blue) consistently achieves the largest coverage area across all three evaluation metrics in both target configurations, indicating superior overall performance. For the top-left corner target, SAL-TD3 achieves a success rate of 85%, representing a 16 percentage point improvement over TD3 (69%). The path length is reduced by 7.7% (from 6.97m to 6.43m), and execution time is decreased by 18.3% (from 93 to 76 s).

For the more challenging top-right corner target requiring longer navigation paths, the performance gap between algorithms becomes more pronounced, as evidenced by the increased separation between algorithm polygons in the right radar chart. SAL-TD3 achieves 84% success rate compared to 66% for TD3 (18 percentage points improvement), with 16.3% reduction in path length (from 8.02 to 6.71 m) and 36.2% reduction in execution time (from 127s to 81s). The bottom panel of Fig. 8 provides a direct quantitative comparison between SAL-TD3 and DDPG, showing improvements of 17.3% in path length (6.57 m vs 7.94 m), 37.2% in execution time (78.5 s vs 125 s), and 24.5 percentage points in success rate (84.5% vs 60%). These substantial improvements demonstrate that the combination of LSTM-based temporal modeling and prioritized experience replay significantly enhances the generalization capability of the path planning algorithm.

Target position	Algorithm	Avg. path length (m)↓	Execution time (s)↓	Success rate (%)↑
Top-Left corner	**SAL-TD3**	**6.43**	**76**	**85**
	L-TD3	6.58	82	81
	P-TD3	6.71	84	79
	TD3	6.97	93	69
	DDPG	7.34	108	62
	SAC	6.89	89	71
Top-Right corner	**SAL-TD3**	**6.71**	**81**	**84**
	L-TD3	6.98	88	80
	P-TD3	7.63	105	77
	TD3	8.02	127	66
	DDPG	8.54	142	58
	SAC	7.56	112	69

Table 4. Generalization performance comparison across different target configurations (100 trials each).

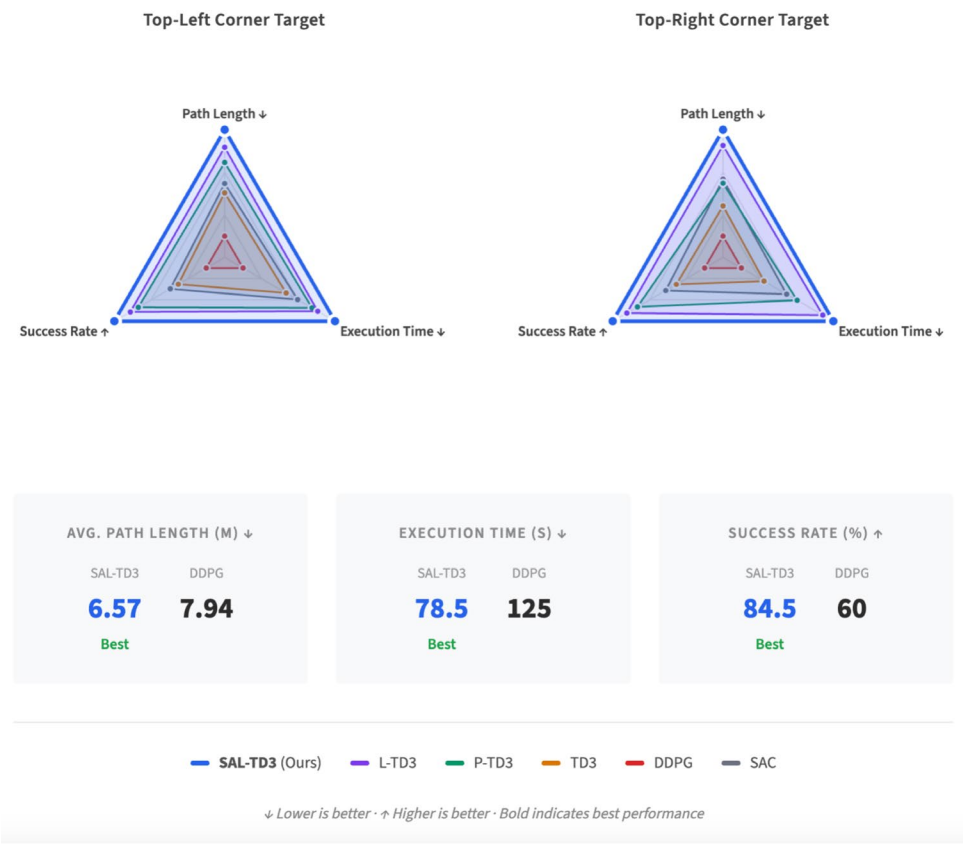


Fig. 8. Generalization performance comparison across algorithms. Top: Radar charts showing multi-dimensional performance metrics for top-left corner target (left) and top-right corner target (right), where larger area indicates better overall performance. Bottom: Direct numerical comparison highlighting SAL-TD3's advantages in average path length, execution time, and success rate against the baseline DDPG algorithm. Arrows indicate optimization direction (↓ lower is better, ↑ higher is better).

A detailed comparison between L-TD3 and P-TD3 in the generalization experiments reveals the complementary nature of these two enhancements. L-TD3 consistently achieves shorter path lengths (6.58m vs 6.71m for top-left, 6.98m vs 7.63m for top-right) and faster execution times compared to P-TD3. In the complex top-right corner scenario with multiple moving obstacles, L-TD3 shortens the average path length by 8.5% and reduces execution time by 16.2% relative to P-TD3. These results underscore the importance of LSTM networks in capturing temporal dependencies for effective decision-making in complex dynamic environments.

Real-world robot experiments

To further validate the practical applicability of SAL-TD3, we deployed the trained model on a physical mobile robot platform and conducted experiments in real indoor environments. The experimental setup and results are presented in this section.

Hardware platform and environment configuration

The real-world experiments utilized an AgileX BUNKER AGV platform featuring a tracked chassis with independent suspension and built-in shock absorbers. The platform dimensions are $1023\text{mm} \times 778\text{mm} \times 400\text{mm}$, equipped with a Robosense RS-Helios-16P LiDAR sensor providing 16-beam 3D point cloud data for precise navigation and obstacle detection. The experimental area measures $8\text{m} \times 8\text{m}$, with cardboard boxes serving as static obstacles and human pedestrians as dynamic obstacles.

The robot's linear velocity range is set to [0, 0.8] m/s, and angular velocity range is $[-1, 1]$ rad/s. A 2D occupancy grid map was constructed using the GMapping SLAM algorithm based on the RBPF (Rao-Blackwellized Particle Filter) approach, which estimates robot pose using LiDAR and odometry data.

Real-world navigation results

Table 5 summarizes the real-world experimental results for SAL-TD3 compared with baseline TD3 and traditional path planning algorithms (A* and RRT). Each algorithm was tested 100 times in both wide area and narrow area scenarios with enforced pedestrian encounters.

The real-world results confirm the effectiveness of SAL-TD3 in practical navigation scenarios. In the wide area scenario, SAL-TD3 achieves an 89% success rate, outperforming TD3 by 13 percentage points, with collision rate reduced from 24% to 11%. The average path length is reduced by 4.4% (from 6.18 to 5.91 m).

In the narrow area scenario, SAL-TD3 maintains excellent performance with 90% success rate and only 10% collision rate, compared to TD3's 74% success rate and 26% collision rate. These results demonstrate that SAL-TD3 exhibits strong adaptability to real-world environmental variations, including sensor noise, localization uncertainty, and unpredictable pedestrian movements.

Traditional planning algorithms (A* and RRT) fail completely in dynamic obstacle scenarios because they assume fixed obstacle configurations during planning and lack mechanisms for real-time detection and avoidance of moving obstacles. Once a path is generated, these algorithms cannot adapt to dynamic environment changes, resulting in inevitable collisions when pedestrians cross the planned trajectory.

Sim-to-real transfer analysis

Comparing simulation results (Tables 2 and 3) with real-world results (Table 5) reveals a modest performance degradation in the sim-to-real transfer. In the wide area scenario, the success rate decreases from 91% (simulation) to 89% (real-world), representing only a 2 percentage point drop. Similarly, in the narrow area scenario, the success rate decreases from 88% to 90%, actually showing slight improvement in real-world conditions.

This robust sim-to-real transfer can be attributed to several factors: (1) the domain randomization applied during training introduces variability in obstacle positions and dynamics, (2) the LSTM-based temporal modeling enables the agent to adapt to different time scales and sensor characteristics, and (3) the prioritized experience replay ensures that the policy is robust to various challenging situations encountered during training.

Ablation study

To investigate the individual contributions of each component in SAL-TD3, we conducted ablation experiments by systematically removing or modifying key algorithmic elements. Table 6 presents the ablation study results in the dynamic environment scenario, and Fig. 9 provides a comprehensive visualization of these results.

As illustrated in Fig. 9, the visualization reveals several important patterns in the ablation results. The bar charts in the top and middle rows clearly demonstrate that the full SAL-TD3 model consistently achieves the best performance across all four metrics. The radar chart in the bottom-left panel provides an intuitive comparison of the multi-dimensional performance trade-offs, where the full model (blue) encompasses a larger area than the ablated variants, indicating superior overall performance. The performance degradation chart in

Scenario	Algorithm	Avg. path length (m)↓	Success rate (%)↑	Collision rate (%)↓
Wide area	**SAL-TD3**	**5.91**	**89**	**11**
	TD3	6.18	76	24
	A*	5.84	0†	100†
	RRT	6.23	0†	100†
Narrow area	**SAL-TD3**	**5.98**	**90**	**10**
	TD3	6.35	74	26
	A*	5.89	0†	100†
	RRT	6.31	0†	100†

Table 5. Real-world navigation performance comparison (100 trials per scenario). Significant values are in [bold]. † A* and RRT algorithms lack dynamic obstacle avoidance capability and result in 100% collision when pedestrian encounters are enforced on the planned path.

Configuration	Metric	Value
SAL-TD3 (Full)	Avg. path length (m)↓	**5.18**
	Execution time (s)↓	**62**
	Success rate (%)↑	**91**
	Collision rate (%)↓	**9**
w/o PER	Avg. path length (m)↓	5.47
	Execution time (s)↓	69
	Success rate (%)↑	86
	Collision rate (%)↓	14
w/o LSTM	Avg. path length (m)↓	5.56
	Execution time (s)↓	74
	Success rate (%)↑	83
	Collision rate (%)↓	17
w/o Self-Attention	Avg. path length (m)↓	5.32
	Execution time (s)↓	66
	Success rate (%)↑	88
	Collision rate (%)↓	12
w/o N-step Returns	Avg. path length (m)↓	5.41
	Execution time (s)↓	68
	Success rate (%)↑	87
	Collision rate (%)↓	13
w/o Layer Norm	Avg. path length (m)↓	5.38
	Execution time (s)↓	67
	Success rate (%)↑	86
	Collision rate (%)↓	14
PER $\alpha = 0.4$	Avg. path length (m)↓	5.36
	Execution time (s)↓	66
	Success rate (%)↑	88
	Collision rate (%)↓	12
PER $\alpha = 0.8$	Avg. path length (m)↓	5.29
	Execution time (s)↓	64
	Success rate (%)↑	89
	Collision rate (%)↓	11

Table 6. Ablation study results in dynamic environment (100 trials). Significant values are in [bold].

the bottom-right panel quantifies the relative importance of each component, highlighting that the removal of N-step returns causes the most significant average degradation (+15.7%), followed by the removal of PER (+19.5% in individual metrics) and LSTM components.

The ablation results provide several key insights:

Impact of PER: Removing the prioritized experience replay mechanism (w/o PER) reduces the success rate from 91% to 86% and increases collision rate from 9% to 14%. The path length increases by 5.6%, confirming that PER significantly enhances sample efficiency and policy quality.

Impact of LSTM: Removing the LSTM component (w/o LSTM) causes the largest performance degradation, with success rate dropping to 83% and collision rate increasing to 17%. This demonstrates the critical role of temporal modeling in dynamic obstacle avoidance, as the agent loses its ability to predict obstacle movements based on historical observations.

Impact of self-attention: The self-attention mechanism contributes moderately to performance, with its removal causing a 3 percentage point drop in success rate. The attention mechanism helps the network focus on relevant temporal segments of the observation history.

Impact of N-step returns: Removing the n-step return computation reduces success rate by 4 percentage points, indicating that multi-step bootstrapping provides more accurate value estimates for navigation tasks with delayed rewards.

Impact of layer normalization: Layer normalization contributes to training stability and final performance, with its removal causing a 5 percentage point drop in success rate.

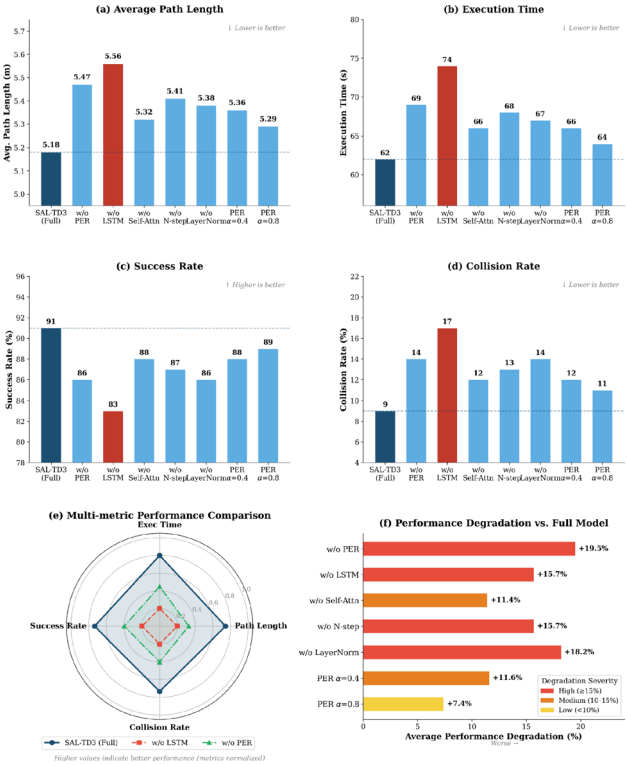


Fig. 9. Visualization of ablation study results. Top row: bar charts comparing average path length and execution time across configurations. Middle row: success rate and collision rate comparisons, with dashed lines indicating the full model baseline. Bottom left: radar chart showing normalized multi-metric performance comparison among SAL-TD3 (Full), w/o LSTM, and w/o PER variants. Bottom right: performance degradation analysis showing the average percentage change across all four metrics relative to the full model, where positive values indicate worse performance.

Algorithm	Training time (h)	Model parameters	Inference time (ms)	GPU memory (GB)
SAL-TD3	98	1.87M	4.2	3.8
L-TD3	92	1.73M	3.9	3.5
P-TD3	85	1.24M	2.8	2.9
TD3	78	1.24M	2.6	2.8
DDPG	65	0.82M	2.1	2.2
SAC	89	1.56M	3.4	3.2

Table 7. Computational efficiency comparison.

PER priority exponent sensitivity: The experiments with different PER priority exponents ($\alpha = 0.4, 0.6, 0.8$) show that $\alpha = 0.6$ achieves the best balance between prioritized sampling and experience diversity. Lower values reduce the prioritization effect, while higher values may cause overfitting to high-priority samples.

Computational efficiency analysis

Table 7 compares the computational requirements of different algorithms during both training and inference phases.

SAL-TD3 requires approximately 98 h of training time, representing a 25.6% increase compared to baseline TD3 (78 h). This additional computational cost stems from the LSTM processing overhead and the priority computation in the experience replay buffer. However, considering the substantial performance improvements (14 percentage points in success rate, 16.6% reduction in path length), this computational overhead is justified for safety-critical mobile robot navigation applications.

The inference time of 4.2 ms for SAL-TD3 remains well within the real-time control requirements (50 ms per control cycle at 20 Hz), ensuring practical deployability on resource-constrained robotic platforms. The model size of 1.87 M parameters is manageable for embedded deployment scenarios.

R_{reach}	R_{collide}	Ratio	Success rate (%)↑	Collision rate (%)↓	Avg. path length (m)↓
500	-400	1.25	85	15	5.52
800	-600	1.33	88	12	5.34
1000	**−800**	**1.25**	**91**	**9**	**5.18**
1000	-1000	1.00	87	8	5.41
1500	-800	1.88	89	13	5.27

Table 8. Sensitivity analysis of terminal reward parameters (wide area, 100 trials) Significant values are in [bold].

Sensitivity analysis of terminal reward parameters

To investigate the influence of the terminal reward magnitudes on policy performance, we conducted a sensitivity analysis by systematically varying the goal-reaching reward R_{reach} and collision penalty R_{collide} while keeping all other hyperparameters fixed. Five representative parameter configurations were evaluated, each tested over 100 trials in the wide area dynamic environment. Table 8 summarizes the results.

The results reveal several important observations. First, reducing both terminal rewards proportionally ($R_{\text{reach}} = 500$, $R_{\text{collide}} = -400$) diminishes the dominance of terminal signals relative to the continuous shaping rewards R_1, R_2, and R_3, causing the agent to under-prioritize both goal reaching and collision avoidance, which leads to a 6 percentage point drop in success rate and a 6 percentage point increase in collision rate compared to the default setting. Second, equalizing the magnitudes ($R_{\text{reach}} = 1000$, $R_{\text{collide}} = -1000$) biases the policy toward overly conservative behavior: while the collision rate decreases slightly to 8%, the agent takes longer and more circuitous paths (5.41 m vs. 5.18 m), and the success rate drops by 4 percentage points because the agent occasionally becomes trapped in local minima to avoid any risk of collision. Third, substantially increasing R_{reach} while keeping R_{collide} unchanged ($R_{\text{reach}} = 1500$, $R_{\text{collide}} = -800$) creates an excessively aggressive policy that prioritizes reaching the goal at the expense of safety, raising the collision rate to 13%. These findings indicate that the default configuration ($R_{\text{reach}} = 1000$, $R_{\text{collide}} = -800$) achieves the best balance between goal-seeking aggressiveness and collision avoidance conservatism. The asymmetric ratio of approximately 1.25 between $|R_{\text{reach}}|$ and $|R_{\text{collide}}|$ encourages the agent to pursue the goal proactively while still maintaining a sufficiently strong safety incentive, which is consistent with the practical requirement that reaching the destination is ultimately more desirable than remaining indefinitely safe but stationary.

Conclusion

This paper proposes the SAL-TD3 algorithm for mobile robot path planning in dynamic environments, integrating LSTM-based temporal modeling, multi-head self-attention, and prioritized experience replay with n-step returns into the TD3 framework. Experimental results demonstrate that SAL-TD3 achieves a 91% success rate (vs. 77% for TD3), reduces path length by 16.6%, and lowers collision rate from 23% to 9%. Generalization tests confirm robust performance in unseen environments (84.5% success rate vs. 60% for DDPG), and real-world deployment validates practical applicability with minimal sim-to-real degradation. In future work, we plan to extend the framework to multi-robot coordination, integrate visual perception modalities such as RGB-D cameras, and investigate transfer learning for rapid adaptation to new environments.

Data availability

The datasets used and/or analysed during the current study are available from the corresponding author on reasonable request. Correspondence should be addressed to You Chen at vxr4175@163.com.

Received: 28 December 2025; Accepted: 23 March 2026
Published online: 10 April 2026

References

1. AbuJabal, N. et al. A comprehensive study of recent path-planning techniques in dynamic environments for autonomous robots. *Sensors* **24**(24), 8089 (2024).
2. Jayalakshmi, K.P., Nair, V.G., & Sathish, D. A comprehensive survey on coverage path planning for mobile robots in dynamic environments. *IEEE Access* (2025).
3. Eirale, A., Martini, M., & Chiaberge, M. Human following and guidance by autonomous mobile robots: a comprehensive review. *IEEE Access* (2025).
4. Adiuku, N. et al. Advancements in learning-based navigation systems for robotic applications in MRO hangar. *Sensors* **24**(5), 1377 (2024).
5. Ghodsian, N., Benfriha, K., & Olabi, A., et al. Mobile manipulators in Industry 4.0: a review of developments for industrial applications. *Sensors* **23**(19), 8026 (2023).
6. Karur, K. et al. A survey of path planning algorithms for mobile robots. *Vehicles* **3**(3), 448–468 (2021).
7. Jones, M., Djahel, S. & Welsh, K. Path-planning for unmanned aerial vehicles with environment complexity considerations: a survey. *ACM Comput. Surv.* **55**(11), 1–39 (2023).
8. Hai, X. et al. Resilient real-time decision-making for autonomous mobile robot path planning in complex dynamic environments. *IEEE Trans. Ind. Electron.* (2025).
9. Debnath, D. et al. A review of uav path-planning algorithms and obstacle avoidance methods for remote sensing applications. *Remote Sens.* **16**(21), 4019 (2024).

10. Hagedorn, S. et al. The integration of prediction and planning in deep learning automated driving systems: a review. *IEEE Trans. Intell. Veh.* (2024).
11. Chib, P. S. & Singh, P. Recent advancements in end-to-end autonomous driving using deep learning: a survey. *IEEE Trans. Intell. Veh.* **9**(1), 103–118 (2023).
12. Hoseinnezhad, R. A comprehensive review of deep learning techniques in mobile robot path planning: categorization and analysis. *Appl. Sci.* **15**(4), 2179 (2025).
13. Houssein, E.H., Mahdy, M.A., & Kayed, M., et al. Integrating soft computing and multi-agent for action recognition: basics, challenging and future directions. *Arch. Comput. Methods Eng.* **2025**, 1–26 (2025).
14. Cheng, N. et al. Prioritized experience replay in path planning via multi-dimensional transition priority fusion. *Front. Neurorobot.* **17**, 1281166 (2023).
15. Li, B. et al. Divide and conquer: advancing large-scale multi-agent pathfinding with hierarchical reinforcement learning. *IEEE Trans. Mob. Comput.* (2025).
16. Chandramoorthy, N. et al. On the generalization of learning algorithms that do not converge. *Adv. Neural Inf. Process. Syst.* **35**, 34241–34257 (2022).
17. Reza, S. et al. A multi-head attention-based transformer model for traffic flow forecasting with a comparative analysis to recurrent neural networks. *Expert Syst. Appl.* **202**, 117275 (2022).
18. Orthey, A., Chamzas, C., & Kavraki, L.E. Sampling-based motion planning: a comparative review. *Annu. Rev. Control Robot. Auton. Syst.* **7**, 256 (2023).
19. Elshaer, A.M., Elmanfaloty, R.A., & Abou-Bakr, E., et al. Exploring algorithmic efficiency of a-star and dijkstra for optimal route planning in green transportation. *Int. J. Intell. Transp. Syst. Res.* **2025**, 1–11 (2025).
20. Wang, J. et al. Trajectory planning for uav navigation in dynamic environments with matrix alignment dijkstra. *Soft Comput.* **26**(22), 12599–12610 (2022).
21. Gao, Y. et al. Improvement and fusion of d* lite algorithm and dynamic window approach for path planning in complex environments. *Machines* **12**(8), 525 (2024).
22. Chu, Y., Chen, Q. & Yan, X. An overview and comparison of traditional motion planning based on rapidly exploring random trees. *Sensors* **25**(7), 2067 (2025).
23. Hüppi, M., Bartolomei, L., & Mascaro, R., et al. T-prm: temporal probabilistic roadmap for path planning in dynamic environments. In *Proc. 2022 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)* 10320–10327 (IEEE, 2022).
24. Zhu, Y. et al. Deep reinforcement learning of mobile robot navigation in dynamic environment: a review. *Sensors* **25**(11), 3394 (2025).
25. Le, H., Saeedvand, S. & Hsu, C. C. A comprehensive review of mobile robot navigation using deep reinforcement learning algorithms in crowded environments. *J. Intell. Rob. Syst.* **110**(4), 158 (2024).
26. Gautier, P., Laurent, J. & Diguet, J. P. Deep q-learning-based dynamic management of a robotic cluster. *IEEE Trans. Autom. Sci. Eng.* **20**(4), 2503–2515 (2022).
27. Sumiea, E.H. et al. Deep deterministic policy gradient algorithm: a systematic review. *Heliyon* **10**, 9 (2024).
28. Zhang, L. et al. A state-decomposition ddpg algorithm for uav autonomous navigation in 3-d complex environments. *IEEE Internet Things J.* **11**(6), 10778–10790 (2023).
29. Xu, C. et al. Enhanced twin delayed ddpg with prioritized experience replay and noisy nets for regional economic dispatch. *Sci. Rep.* (2025).
30. Xu, Y. et al. Multi-objective trajectory optimization method for industrial robots based on improved td3 algorithm. *Sci. Rep.* **15**(1), 41970 (2025).
31. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proc. Int. Conf. Mach. Learn. (ICML), vol. 80* 1861–1870 (PMLR, 2018).
32. Hazem, Z.B., Saidi, F., & Guler, N., et al. Reinforcement learning-based intelligent trajectory tracking for a 5-dof mitsubishi robotic arm: comparative evaluation of ddpg, lc-ddpg, and td3-adx. *Int. J. Intell. Robot. Appl.* **2025**, 1–21 (2025).
33. Zhang, X. et al. Episodic memory governs choices: an rnn-based reinforcement learning model for decision-making task. *Neural Netw.* **134**, 1–10 (2021).
34. Turkoglu, M. O. et al. Gating revisited: deep multi-layer rnns that can be trained. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**(8), 4081–4092 (2021).
35. Papoudakis, G., Christianos, F. & Albrecht, S. Agent modelling under partial observability for deep reinforcement learning. *Adv. Neural Inf. Process. Syst.* **34**, 19210–19222 (2021).
36. Hassani, H. et al. Towards sample-efficiency and generalization of transfer and inverse reinforcement learning: a comprehensive literature review. *IEEE Trans. Artif. Intell.* (2025).
37. Neves, D.E., Ishitani, L., & Patrocinio Junior, Z. K. G. Advances and challenges in learning from experience replay. *Artif. Intell. Rev.* **58**(2), 54 (2024).
38. Hassani, H., Nikan, S. & Shami, A. Improved exploration–exploitation trade-off through adaptive prioritized experience replay. *Neurocomputing* **614**, 128836 (2025).
39. Dou, J. et al. Risk-aware distributional reinforcement learning for safe path planning of surface sensing agents. *Electronics* **14**(24), 4828 (2025).
40. Xu, J., Zheng, Z. & Ren, F. Two-layered reward reinforcement learning in humanoid robot motion tracking. *Mathematics* **13**(21), 3445 (2025).
41. Xu, J. et al. Safecrowdnav: safety evaluation of robot crowd navigation in complex scenes. *Front. Neurorobotics* **17**, 1276519 (2023).
42. Fei, H. et al. Optimizing attention for sequence modeling via reinforcement learning. *IEEE Trans. Neural Netw. Learn. Syst.* **33**(8), 3612–3621 (2021).
43. Hernández, A. & Amigó, J. M. Attention mechanisms and their applications to complex systems. *Entropy* **23**(3), 283 (2021).
44. Wang, T. et al. Multimodal human–robot interaction for human?centric smart manufacturing: a survey. *Adv. Intell. Syst.* **6**(3), 2300359 (2024).
45. Xiao, Z. et al. Time-series forecasting method based on hierarchical spatio-temporal attention mechanism. *Sensors* **25**(13), 4001 (2025).
46. Reza, S. et al. A multi-head attention-based transformer model for traffic flow forecasting with a comparative analysis to recurrent neural networks. *Expert Syst. Appl.* **202**, 117278 (2022).
47. Cao, K., Zhang, T. & Huang, J. Advanced hybrid lstm-transformer architecture for real-time multi-task prediction in engineering systems. *Sci. Rep.* **14**(1), 4890 (2024).
48. Liu, Q. et al. Sample-efficient backtrack temporal difference deep reinforcement learning. *Knowl.-Based Syst.* **2025**, 114613 (2025).
49. Winata, G. I. et al. Preference tuning with human feedback on language, speech, and vision tasks: a survey. *J. Artif. Intell. Res.* **82**, 2595–2661 (2025).
50. Lyle, C. et al. Learning dynamics and generalization in deep reinforcement learning. In *Proc. Int. Conf. Mach. Learn. (ICML)* 14560–14581 (PMLR, 2022).
51. Terven, J. Deep reinforcement learning: a chronological overview and methods. *AI* **6**(3), 46 (2025).

Acknowledgements
During the preparation of this manuscript, the author used ChatGPT (OpenAI, GPT-5) for the purposes of language refinement and formatting suggestions. The author has reviewed and edited the output and takes full responsibility for the content of this publication.

Author contributions
Luyun Chen: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. Qiang Tang: Conceptualization, Methodology, Resources, Writing – review & editing, Supervision. Rui Xu: Validation, Formal analysis, Writing – review & editing. You Chen: Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Project administration. All authors have read and agreed to the published version of the manuscript.

Funding
This research received no external funding.

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to Y.C.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026